PORTFOLIO: REFLECTION & DESIGN DOCUMENT

Phase 2: Reflexions- und Entwurfsbericht zur Softwarearchitektur & OOP

Modul:	Objektorientierte und funktionale Programmierung mit Python (DLBDSOOFPP01_D)
Prüfungsleistung:	Portfolio (Phase 2 Reflexions- und Entwurfsbericht)
Studierender:	Manuel Hanack (Matrikelnummer: IU14156336)
Studiengang:	B.Sc. Cyber Security (2. Fachsemester, Teilzeit 8 Semester)
Datum:	16.10.2026

1. Untersuchung und Diskussion der Umsetzung objektorientierter Konzepte in Python

Der Übergang von einem sprachunabhängigen UML-Entity-Klassendiagramm aus Phase 1 hin zu einer konkreten Implementierung in Python offenbart sprachspezifische Besonderheiten. Im Folgenden wird detailliert untersucht und begründet, wie Kapselung, Datenvalidierung, Enumerations, Komposition und Aggregation im Fachmodell umgesetzt werden.

1.1 Kapselung (Encapsulation) & Notenvalidierung mit Property-Settern

In Python gibt es auf Sprachebene keine harten 'private'-Schlüsselwörter wie in Java oder C++. Kapselung wird stattdessen durch Konventionen (führender Unterstrich _attribute für geschützte Zugriffe) und das Konzept von Properties (@property und @setter) realisiert. Das folgende Codebeispiel aus der Klasse Pruefungsleistung illustriert das Zusammenspiel von geschütztem Attribut, Property-Getter, Property-Setter, Notenvalidierung und abgeleitetem Attribut:

```
class Pruefungsleistung:
    def __init__(self, name: str, ects: int):
        self._name = name
        self._ects = ects
        self._note: Optional[float] = None
        self._bestanden: bool = False  # Abgeleitetes Attribut /bestanden
        self._versuch: int = 1

    @property
    def note(self) -> Optional[float]:
        """Property-Getter: Kontrollierter Lesezugriff."""
        return self._note
```

```
    @note.setter
    def note(self, value: Optional[float]) -> None:
        """Property-Setter: Validierung und Invariantenschutz."""
        if value is None:
            self._note = None
            self._bestanden = False
            return

        erlaubte_noten = {1.0, 1.3, 1.7, 2.0, 2.3, 2.7, 3.0, 3.3, 3.7, 4.0, 5.0}

        matched = False
        for n in erlaubte_noten:
            if abs(value - n) < 1e-5:
                value = n
                matched = True
                break

        if not matched:
            raise ValueError(f"Ungültige Note: {value}. Note muss der Notenskala entsprechen
(z.B. 1.0, 1.3, 1.7 etc.).")

        self._note = value
        self._bestanden = value <= 4.0  # Automatische Aktualisierung des abgeleiteten
Attributs
```

Wirkungsweise im Detail:
1. Geschütztes Attribut (_note): Verhindert den direkten Schreibzugriff von außen.
2. Property-Setter (@note.setter): Fängt jeden Zuweisungsversuch ab und prüft, ob der übergebene Fließkommawert exakt der offiziellen IU-Notenskala entspricht (1.0 bis 4.0 sowie 5.0 bei Nichtbestehen).
3. Fehlerbehandlung (ValueError): Wird eine ungültige Note (z.B. 2.1) übergeben, wirft der Setter sofort eine kontrollierte ValueError-Exception mit einer klaren Fehlermeldung. Der Objektzustand bleibt unverändert.
4. Abgeleitetes Attribut (/bestanden): Das im UML-Diagramm als abgeleitet gekennzeichnete Feld /bestanden wird innerhalb des Setters automatisch auf True gesetzt, wenn value <= 4.0 ist, andernfalls auf False. Dies garantiert jederzeit Datenkonsistenz.

1.2 Komposition vs. Aggregation mit Lebenszyklussteuerung in Python

In Sprachen wie C++ wird Komposition durch direkte Speicherinhalts-Einbettung oder Destruktor-Aufrufe (delete) erzielt. In Python hingegen arbeitet die Speicherverwaltung ausschließlich mit Objektreferenzen und automatischer Garbage Collection (Reference Counting). Eine gespeicherte Objektreferenz alleine gewährleistet daher noch keine Komposition.

Die Unterscheidung zwischen Komposition und Aggregation erfolgt in Python rein über die semantische Implementierung der Lebenszyklussteuerung:

• Komposition (Studiengang ➔ Semester & Modul ➔ Pruefungsleistung):
Ein Semester besitzt eine existentielle Abhängigkeit vom Studiengang. Die Klasse Studiengang erzeugt und verwaltet ihre Semester-Instanzen exklusiv in ihrer internen Liste (self.semester_liste). Wird das Studiengang-Objekt gelöscht oder zurückgesetzt, verlieren alle enthaltenen Semester-Objekte ihre übergeordnete Referenz und werden von Pythons Garbage Collector aus dem Speicher entfernt.

• Aggregation (Semester ➔ Modul):
Module existieren im Fachmodell als eigenständige Entitäten des Curriculums. Ein Semester hält lediglich eine Liste von Referenzen auf Modul-Objekte (self.module). Wird ein Modul aus einem Semester entfernt (z.B. bei einer Umplanung in ein anderes Semester), wird lediglich die Referenz in der Liste gelöscht. Das Modul-Objekt selbst bleibt im Speicher bestehen und behält seine Noten- und Statusdaten bei.

1.3 Refactoring und Überarbeitung des Phase-1-Fachmodells

Gemäß den wissenschaftlichen Vorgaben der Tutoren-Begutachtung aus Phase 1 wurden folgende systematische Modellkorrekturen vorgenommen:

1. Entfernung redundanter Attribute: Die vorher doppelt modellierten Angaben zu ects und note in Modul wurden entfernt. Modul delegiert diese Eigenschaften nun sauber über Properties an die enthaltene Pruefungsleistung.
2. Abgeleitetes Attribut /bestanden: Als abgeleitete Eigenschaft mit Slash (/) deklariert und im Setter dynamisch aus note <= 4.0 ermittelt.
3. Typengerechter Modulstatus (Enum ModulStatus): Der Modulstatus wurde von einem fehleranfälligen Freitextfeld auf eine strikte Enumeration (ModulStatus.OFFEN, ModulStatus.IN_BEARBEITUNG, ModulStatus.BESTANDEN) umgestellt.
4. ECTS-Soll-Fortschrittskontrolle: Die Methode get_ects_soll(semester) berechnet dynamisch das ECTS-Soll (45 ECTS im 2. Semester bei 22,5 ECTS/Semester) ohne unnötige Datumsfelder.
5. Bereinigung redundanter Assoziations-Attribute: Attribute wie semester_liste oder module wurden aus den Klassenkästen des UML-Diagramms entfernt, da sie bereits durch Assoziationslinien ausgedrückt werden.

1.4 Relationales O/R-Mapping & Rekonstruktion der Objektstruktur in SQLite

Die Speicherung der Objektstruktur erfolgt relational in einer lokalen SQLite-Datenbank (dashboard.db) über die Klasse DatabaseManager. Das Datenbankschema umfasst zwei Tabellen:
• student_info: Speichert Stammdaten (Name, Matrikelnummer, Studiengang, Regelsemester).
• module_progress: Speichert Primärdaten der 29 Module (code [PK], name, ects, semester, note, status, versuch).

Rekonstruktion des Objektbaums (Object Graph Hydration):
Beim Anwendungsstart führt die Methode DatabaseManager.load_studiengang() das O/R-

Mapping aus und verbindet die flachen Datenbankzeilen wieder zu einem vollständigen Fachobjekt-Baum:

```
def load_studiengang(self) -> Studiengang:
    # 1. Haupt-Aggregat instanziieren
    info = self.get_student_info()
    studiengang = Studiengang(name=info['studiengang'], gesamt_ects_ziel=180,
regelsemester=info['regelsesemester'])

    # 2. Semester-Kompasse (1 bis 6) erzeugen und hinzufügen
    semester_dict = {num: Semester(num) for num in range(1, 7)}
    for sem in semester_dict.values():
        studiengang.add_semester(sem)

    # 3. Datenbankzeilen abfragen und zu Objektstrukturen verknüpfen
    with self.get_connection() as conn:
        rows = conn.cursor().execute("SELECT * FROM module_progress ORDER BY
semester, code;").fetchall()
        for r in rows:
            pl = Pruefungsleistung(name=f"Prüfung {r['name']}", ects=r['ects'])
            if r['note'] is not None:
                pl.note = r['note']
            pl.versuch = r['versuch']

            status_enum = ModulStatus[r['status'].upper().replace(" ", "_")]
            modul = Modul(code=r['code'], name=r['name'], pruefungsleistung=pl,
status=status_enum)

            semester_dict[r['semester']].add_modul(modul)

    return studiengang
```

2. Gesamtarchitektur (MVC-Muster mit explizitem Controller) & UML-Diagramm

Gemäß dem Tutor-Feedback wurde die Architektur strikt nach dem Model-View-Controller (MVC) Muster entworfen und eine explizite Controller-Klasse (DashboardController) eingeführt. Dadurch sind Benutzeroberfläche, Steuerungslogik und Datenzugriff vollständig entkoppelt.

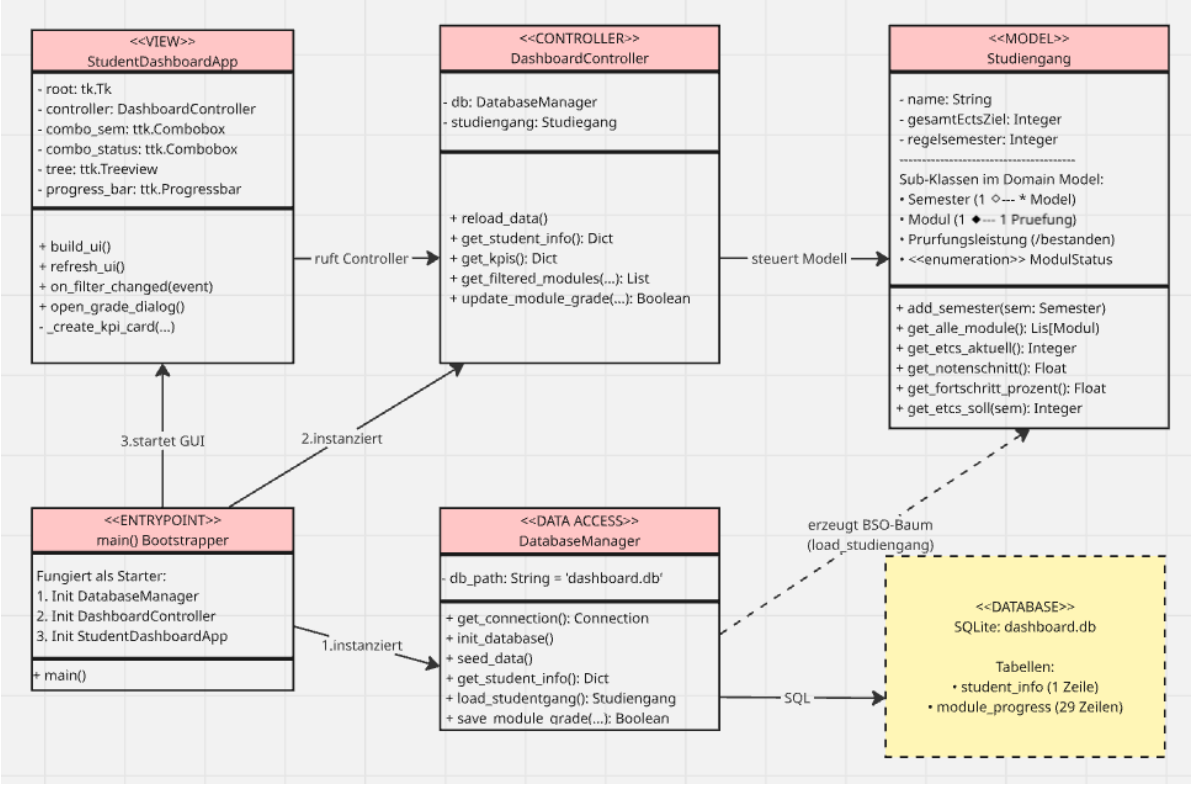


Abbildung 3: Vollständiges UML-Systemarchitekturdiagramm mit explizitem Controller, View, Domain Model und Data Access Layer

Beschreibung der Systemschichten im UML-Architekturdiagramm:

1. View Layer (gui.py -> StudentDashboardApp):
Reine Präsentationskomponente auf Tkinter-Basis. Sie baut die Oberfläche auf (KPI-Cards, Fortschrittsbalken, Treeview, Filter, Dialoge) und leitet sämtliche Benutzeraktionen ohne eigene Geschäftslogik direkt an den DashboardController weiter.
2. Controller Layer (controller.py -> DashboardController):
Explizite Steuerungsklasse. Sie nimmt Anfragen der View entgegen (z.B. update_module_grade, get_kpis, get_filtered_modules), steuert das in-memory Fachmodell (Studiengang) und beauftragt den DatabaseManager mit der dauerhaften Speicherung.
3. Domain Model Layer (models.py -> Studiengang, Semester, Modul, Pruefungsleistung, ModulStatus):
Frei von GUI- und SQL-Code. Kapselt die Geschäftslogik, Notenvalidierung und Berechnungen (Notenschnitt, ECTS-Fortschritt).
4. Data Access Layer / DAL (database.py -> DatabaseManager):
Kapselt relationale SQLite-Operationen (dashboard.db), führt das automatisierte Schema-Building und Seeding durch und stellt mit load_studiengang() und save_module_grade() klare Datenzugriffsoperationen bereit.